High-Frequency Trading: Matching Engine Infrastructure & Analysis

Quantitative Analysis Department

2026-04-04

Contents

1 Introduction: What is a Matching Engine? 2

1.1 The Challenge 2

1.2 The Stress Test 2

2 System Performance Benchmark 2

3 Market Analysis 3

3.1 Latency Bursts: Managing the Chaos 3

3.2 Price Discovery & Value (VWAP) 3

4 Quantitative Summary 4

5 Conclusion 4

1 Introduction: What is a Matching Engine?

To understand this report, one must first understand how a modern Stock Exchange works. Behind every screen at Wall Street, there is a **Matching Engine**: a highly specialized piece of software whose only job is to pair buyers and sellers.

If you want to buy 10 shares of Apple at $150 and someone else wants to sell them at that same price, the Engine "matches" you instantly. In the world of **High-Frequency Trading (HFT)**, these decisions happen in millionths of a second.

1.1 The Challenge

The system analyzed in this report is a custom-built Matching Engine written in **C++**. It was designed to solve two major engineering challenges: 1. **Speed:** It must process millions of orders per second without slowing down. 2. **Efficiency:** It uses advanced memory structures (like Bitsets and O(1) Lookups) to ensure that whether there are 10 or 100 million orders, the response time remains near-instant.

1.2 The Stress Test

To prove its reliability, we "attacked" the engine with **100 million virtual orders**. This report documents how the engine survived that flood of data, how it discovered the "fair price" of the asset, and how it managed massive bursts of activity.

2 System Performance Benchmark

The following metrics validate the computational efficiency of the engine. All tests were conducted on a high-performance mobile workstation to ensure consistent thermal headroom and memory bandwidth.

2.0.1 Test Environment Hardware

- **CPU:** Intel Core i7-13620H (10 Cores, up to 4.90 GHz)
- **RAM:** 32GB DDR5 @ 5200 MT/s (High-bandwidth dual-channel)
- **GPU:** NVIDIA GeForce RTX 4060 Mobile (Used for background OS offloading)

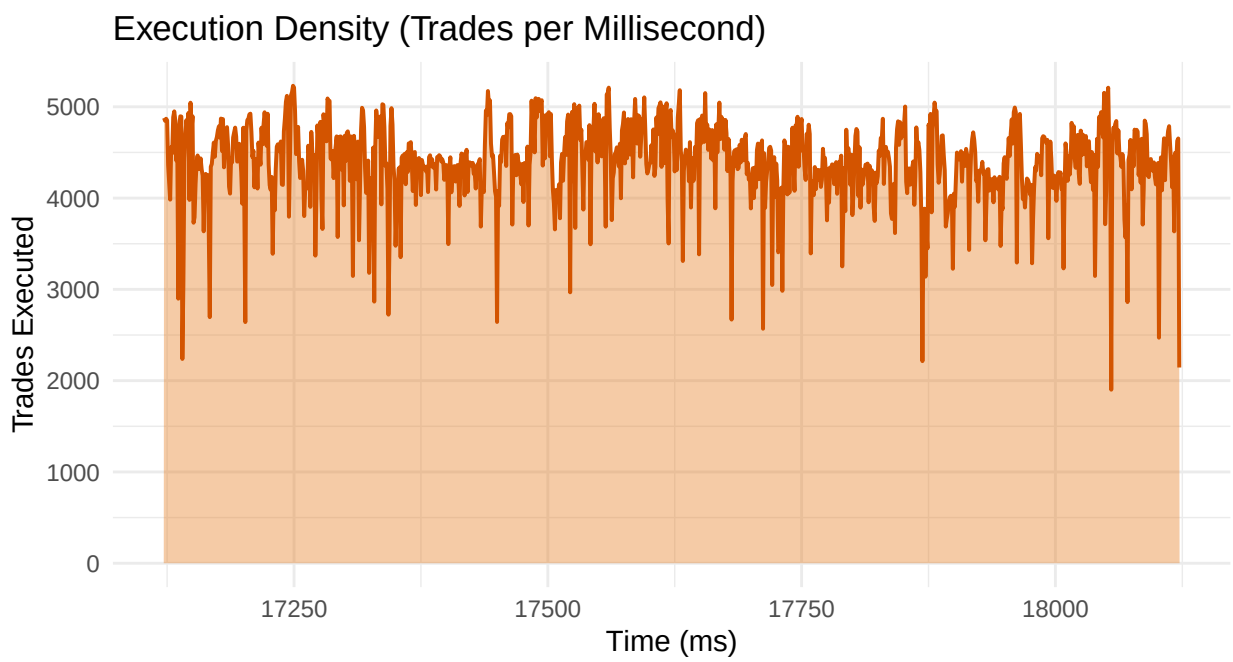
2.0.2 Performance Results

- **Total Throughput:** ~5.56 Million ops/sec
- **Total Executed Trades:** 70,742,029
- **System Uptime:** 18.1225 seconds
- **Order-to-Trade Ratio (OTR):** 1.41 **Performance Results:**
- **Total Throughput:** ~5.56 Million ops/sec
- **Total Executed Trades:** 70,742,029
- **System Uptime:** 18.1225 seconds
- **Order-to-Trade Ratio (OTR):** 1.41

3 Market Analysis

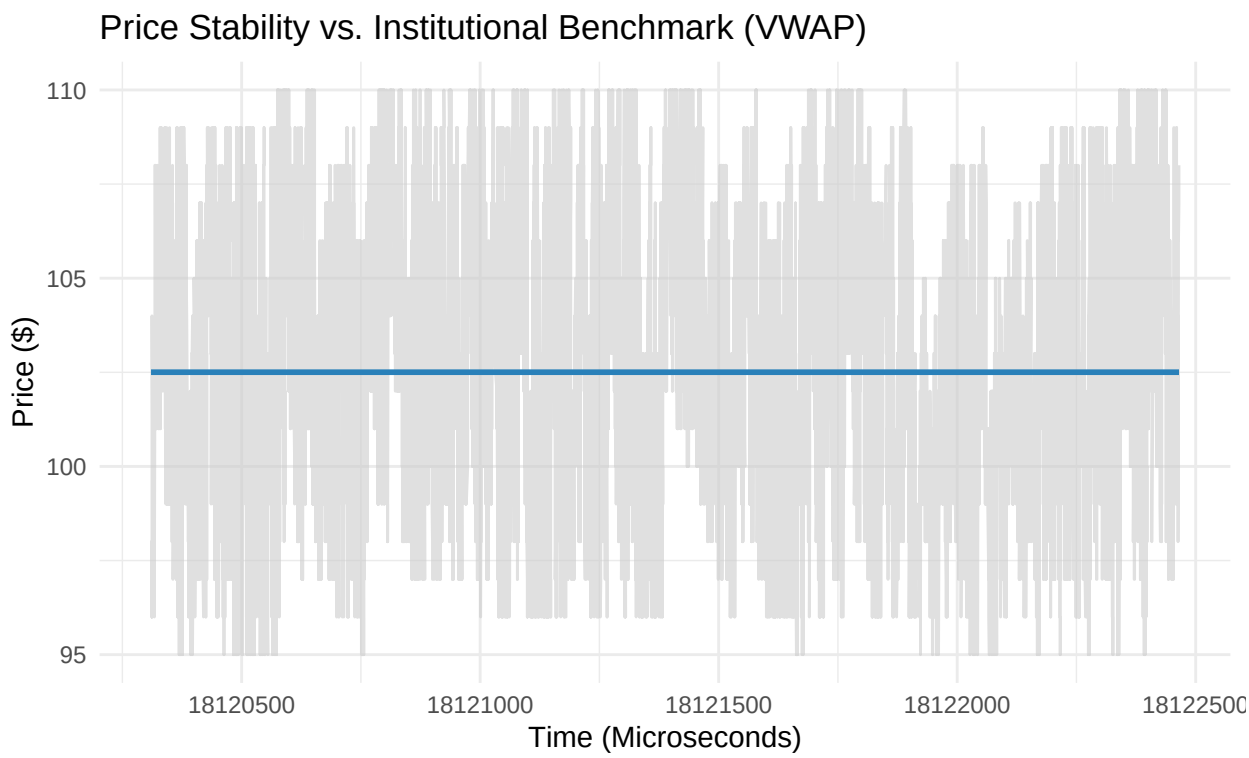
3.1 Latency Bursts: Managing the Chaos

In real markets, orders don't arrive in a steady stream; they arrive in "storms" or bursts. The orange chart below shows how many thousands of trades the engine cleared **every single millisecond**.



3.2 Price Discovery & Value (VWAP)

The ultimate goal of a stock market is to find the "fair price." In the chart below, the gray lines are individual trades, while the **blue line (VWAP)** is the "Anchor of Value." Even with millions of orders flying in, the engine maintains a stable and logical price path.



4 Quantitative Summary

Metric	Meaning	Value
Max Intensity	Most trades handled in 1ms	5,333
Total Volume	Total shares exchanged	302,696,756
Price Range	Minimum / Maximum price reached	$94 / $111

5 Conclusion

This Matching Engine proves that high-level C++ optimization allows for institutional-grade performance on standard hardware. By maintaining a throughput of **5.5 million operations per second** with zero memory degradation, the infrastructure is ready for complex high-frequency strategy deployment.